Mitigação Adaptativa de Ataques DDoS em Redes Definidas por Software com Aprendizado por Reforço

Bernardo Euler Barbosa Santos (567068)[1]
Renato Rogério Barros Martins (561497)[1]

[1]Universidade Federal do Ceará (UFC) – Campus de Russas – Russas, CE – Brasil

Abstract. *In Software-Defined Networks (SDN), static DDoS defenses either block legitimate users or react too slowly. This paper proposes a deep reinforcement learning agent that, for each traffic source, chooses among four graduated actions: allow, rate-limit (OpenFlow meters), redirect to an analysis host, or temporarily block. The reward penalizes collateral damage to legitimate users and the cost of each action. The agent was implemented on Mininet, Open vSwitch and os-ken (a maintained Ryu fork), trained in a simulator calibrated with emulation traces, and compared with fixed thresholds, entropy detection and Random Forest classifiers. Preliminary results (one seed) show that the agent reacts fastest (0.3 s) at negligible controller cost and, mostly by redirecting attackers, keeps availability above 99% under UDP and ICMP floods; under SYN floods, however, it restricts many legitimate sources and trails fixed thresholds and Random Forest.*

Resumo. *Em Redes Definidas por Software (SDN), defesas estáticas contra ataques DDoS ou bloqueiam usuários legítimos ou reagem tarde demais. Este artigo propõe um agente de aprendizado por reforço profundo que, para cada origem de tráfego, escolhe entre quatro ações graduadas: permitir, limitar a taxa (meters OpenFlow), redirecionar para um nó de análise ou bloquear temporariamente. A recompensa penaliza o dano colateral a usuários legítimos e o custo de cada ação. O agente foi implementado sobre Mininet, Open vSwitch e os-ken (fork mantido do Ryu), treinado em um simulador calibrado com traços da emulação e comparado a limiares fixos, detecção por entropia e classificadores Random Forest. Resultados preliminares (uma semente) mostram que o agente reage mais rápido (0,3 s), com custo desprezível no controlador, e, sobretudo redirecionando os atacantes, mantém a disponibilidade acima de 99% sob inundações UDP e ICMP; sob inundação SYN, porém, restringe muitas origens legítimas e fica atrás do limiar fixo e do Random Forest.*

1. Introdução

Ataques distribuídos de negação de serviço (*Distributed Denial of Service* – DDoS) buscam esgotar recursos de rede ou de processamento de um alvo por meio de grande volume de tráfego originado de múltiplas fontes, tornando serviços indisponíveis a usuários legítimos [Mirkovic and Reiher 2004]. Apesar de mais de duas décadas de pesquisa, a mitigação em tempo real permanece difícil porque o tráfego de ataque se mistura ao tráfego legítimo [Liu et al. 2018].

As Redes Definidas por Software (*Software-Defined Networking* – SDN) separam o plano de controle do plano de dados e oferecem ao controlador uma visão global da rede

[McKeown et al. 2008, Kreutz et al. 2015]. Essa característica permite coletar estatísticas de fluxos e instalar regras de descarte, limitação de taxa ou redirecionamento de forma programática. Por outro lado, a própria SDN introduz novas superfícies de ataque, como a saturação do controlador por mensagens *packet-in* e o esgotamento das tabelas de fluxo dos *switches* [Yan et al. 2016, Albuquerque Jr. and Fonseca 2023].

Problema. Mecanismos de defesa baseados em regras fixas enfrentam um dilema: um limiar rígido bloqueia usuários legítimos em picos de demanda (*flash crowds*), enquanto um limiar permissivo ou uma reação lenta permite a continuidade do ataque. Grande parte da literatura recente concentra-se na *detecção* supervisionada, frequentemente com classificadores como Random Forest [Bahashwan et al. 2023, Silva et al. 2024, Dias et al. 2023], e trata a *resposta* de forma secundária, muitas vezes como uma decisão binária de bloquear ou não.

Quem é afetado. Usuários legítimos do serviço atacado, que sofrem indisponibilidade ou bloqueio indevido; operadores de rede, que precisam calibrar regras manualmente; e o próprio controlador SDN, que pode ser sobrecarregado durante o ataque.

Escopo. Considera-se uma rede SDN de um único domínio, com um controlador e topologia em árvore emulada no Mininet, protegendo um servidor HTTP contra ataques volumétricos (*SYN*, UDP e ICMP *flood*), inclusive pulsados e multivetor. Ataques à camada de aplicação, ataques com falsificação de endereços de origem e cenários com múltiplos controladores ficam fora do escopo.

Objetivo geral. Desenvolver e avaliar um agente de aprendizado por reforço (*Reinforcement Learning* – RL) que observe o tráfego em uma rede SDN e aprenda, para cada origem de tráfego, quando *permitir*, *limitar a taxa*, *redirecionar para análise* ou *bloquear temporariamente*, reduzindo o ataque sem bloquear usuários legítimos.

Objetivos específicos.

1. Construir um ambiente emulado reprodutível com Mininet, Open vSwitch e controlador os-ken (*fork* mantido do Ryu), com geração automatizada de tráfego legítimo e de ataque.
2. Formular a mitigação como um Processo de Decisão de Markov (MDP), com estados, ações graduadas e recompensa que penaliza dano colateral.
3. Treinar um agente *Double DQN* e integrá-lo ao controlador.
4. Comparar o agente com regras de limiar fixo, detecção por entropia e detectores supervisionados, sob as mesmas condições, quanto a tráfego de ataque bloqueado, falsos positivos, disponibilidade, latência, tempo de reação e sobrecarga no controlador.

Contribuições. (i) Uma formulação de mitigação com *espaço de ações graduado* de quatro níveis, aplicado por origem e com política compartilhada, que escala com o número de fontes; (ii) uma função de recompensa que equilibra explicitamente eficácia, dano a usuários legítimos e custo operacional das ações; (iii) uma avaliação que inclui cenários adversos não vistos no treinamento – ataques pulsados, *flash crowds* e enlaces com atraso e perda –, verificando se a resposta se adapta à intensidade do ataque e às condições da rede, e que mede a sobrecarga imposta ao controlador; e (iv) uma implementação aberta e reprodutível – controlador, simulador calibrado, agente, *baselines* e *scripts* de experimento –, disponível no repositório do projeto.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação; a Seção 3 discute os trabalhos relacionados; a Seção 4 descreve a metodologia; a Seção 5 detalha a solução proposta; a Seção 6 apresenta o planejamento experimental; as Seções 7 e 8 trazem resultados, discussão e ameaças à validade; e a Seção 9 conclui o trabalho.

2. Fundamentação

2.1. Redes Definidas por Software e OpenFlow

Em SDN, a lógica de controle é retirada dos equipamentos de encaminhamento e centralizada em um controlador, que programa os *switches* por meio de uma interface padronizada, tipicamente o OpenFlow [McKeown et al. 2008, Kreutz et al. 2015]. Cada *switch* mantém tabelas de fluxo cujas entradas possuem campos de casamento (por exemplo, endereços IP, portas e protocolo), prioridade, contadores e instruções. Na versão 1.3 do OpenFlow [Open Networking Foundation 2012], destacam-se três recursos usados neste trabalho:

- **Contadores e estatísticas**: o controlador solicita periodicamente estatísticas por fluxo e por porta (*FlowStats*/*PortStats*), obtendo pacotes e bytes acumulados.
- **Temporizadores**: entradas podem expirar por inatividade (*idle_timeout*) ou após um tempo fixo (*hard_timeout*), o que permite bloqueios temporários que se desfazem automaticamente.
- **Tabela de medidores (*meters*)**: um medidor com banda do tipo *drop* descarta pacotes que excedem uma taxa configurada, implementando limitação de taxa por fluxo ou por origem.

O Open vSwitch (OVS) é um *switch* virtual de código aberto com suporte a OpenFlow [Pfaff et al. 2015]; o Mininet emula redes inteiras com *hosts*, *switches* e enlaces em uma única máquina [Lantz et al. 2010]; e o Ryu é um arcabouço em Python para construção de controladores [Ryu SDN Framework Community 2020].

2.2. Ataques DDoS

Ataques DDoS podem ser classificados quanto ao recurso explorado e à dinâmica da taxa de envio [Mirkovic and Reiher 2004]. Os ataques de inundação (*flooding*) volumétricos mais comuns são: *TCP SYN flood*, que esgota a fila de conexões semiabertas do servidor; *UDP flood* e *ICMP flood*, que saturam a banda do enlace da vítima. Em ataques de baixa taxa ou pulsados, o atacante alterna rajadas e silêncio para degradar o serviço evitando detectores baseados em médias [Kuzmanovic and Knightly 2003]. Em SDN, há ainda ataques ao plano de controle: pacotes com cabeçalhos aleatórios geram *packet-in* em excesso e sobrecarregam o controlador, ou preenchem a memória TCAM dos *switches* [Yan et al. 2016, Albuquerque Jr. and Fonseca 2023].

Atributos clássicos para caracterizar tráfego de ataque incluem estatísticas de fluxo, como pacotes e bytes por fluxo, a proporção de fluxos sem resposta (assimetria entre ida e volta) e o crescimento do número de portas [Braga et al. 2010], além da entropia dos endereços de destino, que cai quando o tráfego converge para uma vítima [Mousavi and St-Hilaire 2015]. Também são comuns a taxa de pacotes por origem, o tamanho médio dos pacotes e a proporção de segmentos SYN. Bases públicas como a CIC-DDoS2019 [Sharafaldin et al. 2019] e a InSDN [Elsayed et al. 2020] são amplamente usadas para avaliar detectores.

2.3. Aprendizado por Reforço

No RL, um agente interage com um ambiente modelado como um MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: em cada passo t ele observa o estado s_t, executa a ação a_t, recebe a recompensa r_t e o ambiente transita para s_{t+1} [Sutton and Barto 2018]. O objetivo é aprender uma política π que maximize o retorno descontado $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$. O *Q-learning* estima a função ação-valor $Q(s, a)$ por diferenças temporais [Watkins and Dayan 1992]. A *Deep Q-Network* (DQN) aproxima Q com uma rede neural, utilizando memória de repetição e rede-alvo [Mnih et al. 2015]. O *Double DQN* reduz a superestimação dos valores ao separar a seleção da ação da sua avaliação [van Hasselt et al. 2016], e a arquitetura *dueling* decompõe Q em valor do estado e vantagem da ação [Wang et al. 2016]. Métodos de gradiente de política, como o PPO [Schulman et al. 2017], são alternativas para espaços de ação maiores.

2.4. Termos técnicos adotados

- **Origem**: endereço IP de origem observado na borda da rede; é a unidade sobre a qual o agente decide.
- **Intervalo de decisão** (Δt): período entre duas coletas de estatísticas e, portanto, entre duas decisões do agente.
- **Ação restritiva**: limitar, redirecionar ou bloquear uma origem.
- **Falso positivo**: ação restritiva aplicada a uma origem legítima.
- **Dano colateral**: tráfego legítimo perdido, atrasado ou rejeitado em consequência das ações de mitigação.

3. Trabalhos Relacionados

Detecção supervisionada em SDN. Revisões sistemáticas recentes mostram que a maior parte dos trabalhos sobre DDoS em SDN trata de detecção com aprendizado de máquina e aprendizado profundo, com abordagens de classificador único, híbridas e de *ensemble* [Bahashwan et al. 2023, Tebbaa et al. 2025]. No contexto nacional, [Silva et al. 2024] avaliam cinco algoritmos na base CIC-DDoS2019, com destaque para o Random Forest, e [Dias et al. 2023] propõem uma arquitetura de microsserviços para detecção em SDN que permite usar modelos mais complexos sem prejudicar o desempenho do controlador. Esses trabalhos alcançam alta acurácia, mas a resposta ao ataque não é o foco, e a decisão resultante costuma ser binária. Trabalhos anteriores à popularização do aprendizado profundo já exploravam mapas auto-organizáveis [Braga et al. 2010] e entropia de endereços de destino [Mousavi and St-Hilaire 2015] para detecção leve no controlador.

RL para resposta a DDoS. [Malialis and Kudenko 2013] propõem o *Multiagent Router Throttling*, em que roteadores a montante aprendem a limitar o tráfego destinado à vítima, com extensão posterior voltada à escalabilidade para um número maior de agentes [Malialis and Kudenko 2015]. [Liu et al. 2018] aplicam DRL em SDN para aprender políticas de limitação contra inundações SYN, UDP e ICMP, superando um método de *router throttling*. O *DeepThrottle* usa DRL para ajustar a limitação em SDN, aumentando a fração de tráfego normal que chega ao servidor em cenários de ataque não vistos [Chen et al. 2022], e [Shen et al. 2023] propõem recompensa com punição adaptativa e alocação de banda para preservar tráfego normal em SDN-IoT. [Feng et al. 2020] tratam ataques na camada de aplicação com uma recompensa multiobjetivo: o agente

mitiga agressivamente quando o servidor está sobrecarregado, mesmo sacrificando algumas requisições legítimas, e de forma conservadora nos demais casos, para minimizar o dano colateral – uma ideia de resposta proporcional próxima da investigada aqui, porém fora do contexto de SDN. [Yungaicela-Naula et al. 2022] combinam um IDS baseado em aprendizado profundo com um IPS baseado em DRL contra ataques de baixa taxa em Mininet e ONOS, estendendo a abordagem com NFV e defesa de alvo móvel [Yungaicela-Naula et al. 2023]. [Khozam et al. 2024] empregam DRL para mitigar inundações ICMP, SYN e UDP preservando QoS em SDN para 5G, e o *QoSentry*, dos mesmos autores, utiliza *Double DQN* com contramedidas adaptáveis, como limitação e redirecionamento de tráfego, e recompensa orientada à qualidade de serviço percebida por usuários legítimos (latência e *jitter*) [Khozam et al. 2025]. Por fim, o *MininetGym* oferece um ambiente modular baseado em Mininet para treinar agentes DQN, PPO e A2C em cenários de segurança [Finistrella et al. 2025].

A Tabela 1 sintetiza os trabalhos mais próximos.

Tabela 1. Quadro comparativo dos trabalhos relacionados (n.i.: não informado no material consultado; n.a.: não se aplica).

Trabalho	Técnica	Ações de resposta	Ambiente / base	Métricas principais	Custo no controlador
Malialis e Kudenko (2013; 2015)	RL multiagente	Limitação em roteadores a montante	Simulação (não SDN)	Tráfego legítimo que alcança o servidor	Não
Liu et al. (2018)	DRL	Limitação de tráfego	SDN; SYN, UDP e ICMP	Desempenho frente a *router throttling*	n.i.
Feng et al. (2020)	RL	Mitigação de requisições (camada 7)	Servidor web (não SDN)	Mensagens maliciosas mitigadas; dano colateral	Não
Yungaicela-Naula et al. (2022)	DL + DRL	Mitigação de fluxos de baixa taxa	Mininet + ONOS	Taxa de detecção; sucesso da mitigação	Parcial (amostragem de fluxos)
Chen et al. (2022)	DRL	Limitação de tráfego	SDN	Tráfego normal entregue; carga do servidor	n.i.
Shen et al. (2023)	DRL	Limitação + alocação de banda	SDN-IoT	Tráfego normal encaminhado; convergência	n.i.
Khozam et al. (2024; 2025)	Double DQN	Limitação e redirecionamento	SDN (5G); SYN, UDP e ICMP	QoS de usuários legítimos (latência, *jitter*)	n.i.
Silva et al. (2024)	Supervisionado (RF e outros)	Somente detecção	CIC-DDoS2019	Acurácia, F1	n.a.
Dias et al. (2023)	Supervisionado (RF e outros)	Somente detecção	SDN com microsserviços	F1; desempenho do controlador	Parcial
Este trabalho	Double DQN	Permitir, limitar (*meter*), redirecionar, bloquear temporariamente	Mininet + OVS + os-ken	Ataque bloqueado, falsos positivos, disponibilidade, latência, reação	Sim

Lacuna. Nos trabalhos analisados, a resposta aprendida é, em geral, *unidimensional* (um grau de limitação) ou binária (bloquear ou não); a exceção mais próxima é o *QoSentry* [Khozam et al. 2025], que combina limitação e redirecionamento. Ainda assim, poucos estudos reúnem, em uma mesma avaliação, (i) um conjunto de ações de naturezas diferentes – limitação, redirecionamento para inspeção e bloqueio reversível –, (ii) cenários projetados para provocar falsos positivos, como *flash crowds* legítimos, e (iii) a

medição do custo que a própria política impõe ao controlador. Este trabalho não pretende reproduzir uma abordagem existente; ele investiga se um espaço de ações graduado, com custos explícitos por ação, produz respostas proporcionais à intensidade do ataque e reduz o dano colateral em relação a regras fixas e a detectores supervisionados.

4. Metodologia

O trabalho segue uma metodologia experimental em etapas, alinhada ao protocolo proposto para a disciplina:

1. **Delimitação**: definição do problema (mitigação de DDoS volumétrico contra um serviço HTTP em uma rede SDN de um único domínio), do escopo e das contribuições (Seção 1).
2. **Fundamentação e revisão**: estudo de SDN/OpenFlow, ataques DDoS e RL, e levantamento de trabalhos por meio de buscadores acadêmicos e de metadados (Crossref, Semantic Scholar) cobrindo publicações da IEEE, ACM, Elsevier, Springer, AAAI e da SOL/SBC (SBRC, SBSeg, WGRS), com as palavras-chave *DDoS*, *SDN*, *reinforcement learning*, *mitigation* e equivalentes em português, priorizando o período 2018–2026 (Seções 2 e 3).
3. **Planejamento**: formulação de questões de pesquisa, hipóteses, *baselines*, métricas e cenários (Seção 6).
4. **Ambiente e dados**: construção da topologia emulada, coleta automatizada de tráfego rotulado, produzido de forma autorizada em ambiente isolado, e automação do registro de *logs* e métricas. Foram usados apenas dados gerados na própria emulação: bases públicas como a CIC-DDoS2019 [Sharafaldin et al. 2019] e a InSDN [Elsayed et al. 2020] não foram adotadas porque seus atributos, extraídos de capturas completas, não correspondem aos contadores e às amostras sFlow disponíveis ao controlador.
5. **Implementação-base**: controlador com monitoramento, regras de limiar fixo e detector supervisionado; 12 testes unitários (parser sFlow, atributos, recompensa, simulador, paridade entre as implementações do agente e *baselines*), um teste de fumaça do OVS e rodadas curtas de integração.
6. **Método proposto**: implementação do ambiente de treinamento, do agente e da sua integração com o controlador, com registro de hiperparâmetros e auditoria de cada decisão.
7. **Experimentos e análise**: execução de todos os métodos nas mesmas condições, com repetições, preservação dos dados brutos e testes estatísticos.

Ambiente computacional. Os experimentos foram executados em Ubuntu 26.04 sobre WSL2 (núcleo 6.18, Windows 10, 12 núcleos lógicos e 8 GB de RAM), com Mininet 2.3.0, Open vSwitch 3.7.1 (OpenFlow 1.3, *datapath* do núcleo), os-ken 4.2.2 – o *fork* mantido do Ryu [Ryu SDN Framework Community 2020], de API compatível, adotado porque o Ryu 4.34 não é mantido desde 2020 e é incompatível com Python recente –, Python 3.14, PyTorch 2.14, Gymnasium 1.4 [Towers et al. 2025] e scikit-learn 1.9. O núcleo do WSL2 já inclui como módulos o `openvswitch` e as disciplinas de fila usadas pelo Mininet (HTB, *netem*, TBF), dispensando recompilação. Os ataques foram gerados com `hping3`, que atinge taxas de milhares de pacotes por segundo com menor custo de CPU que geradores em Python como o Scapy [Biondi and Scapy Community 2026]. Todos os parâmetros e sementes estão em um arquivo de configuração versionado junto ao código.

5. Arquitetura e Solução Proposta

5.1. Visão geral da arquitetura

A Figura 1 apresenta a arquitetura. O plano de dados, emulado no Mininet, contém clientes legítimos, atacantes, *switches* OVS, o servidor-vítima (HTTP) e um nó de análise. No plano de controle, a aplicação os-ken é composta por quatro módulos: (i) **monitor**, que coleta *FlowStats*, *PortStats* e amostras sFlow a cada Δt; (ii) **extrator de atributos**, que agrega as estatísticas por origem; (iii) **agente**, que escolhe uma ação para cada origem ativa; e (iv) **atuador**, que traduz a ação em mensagens OpenFlow. Toda decisão é registrada com estado observado, valores Q e ação escolhida, permitindo auditoria *a posteriori*. Para explicar as decisões, calcula-se a importância de cada atributo por permutação sobre esses registros, medindo quanto a ação escolhida muda quando o atributo é embaralhado.

Coleta de atributos. Contadores OpenFlow fornecem apenas pacotes e bytes por regra. Por isso, a tabela 0 de cada *switch* de borda mantém uma entrada por endereço ativo como origem e outra como destino, que contam o tráfego e recebem as ações de mitigação, seguidas do encaminhamento na tabela 1. Atributos que dependem de cabeçalhos (proporção de SYN, portas de destino distintas e entropia de endereços) são obtidos por amostragem sFlow, suportada nativamente pelo OVS, com um coletor integrado ao controlador.

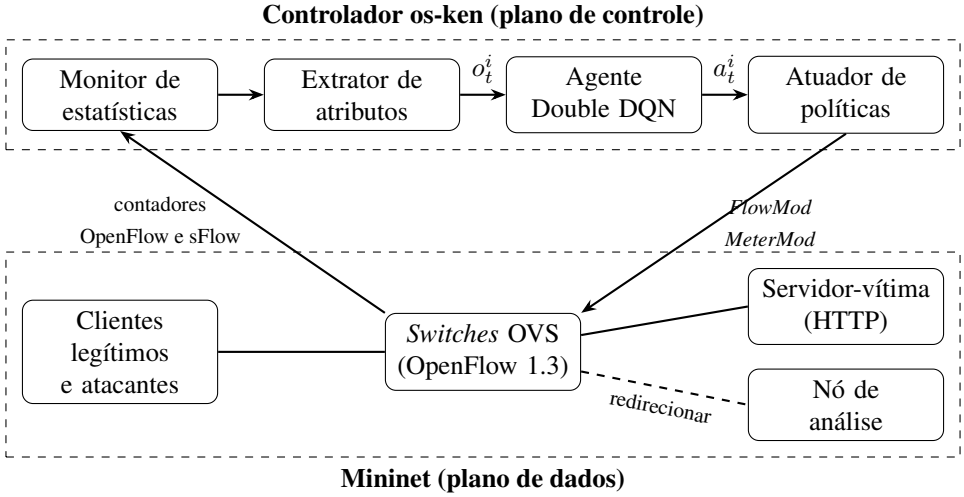


Figura 1. Arquitetura da solução proposta.

5.2. Topologia

A topologia base é uma árvore de dois níveis: um *switch* de núcleo conectado ao servidor-vítima, ao nó de análise e a um *host* de monitoramento, e três *switches* de borda com quatro *hosts* cada. Entre os 12 *hosts*, a proporção entre legítimos e atacantes varia por cenário. Os enlaces são configurados com `TCLink` (100 Mbit/s, 0,5 ms, filas de 100 pacotes), e o enlace da vítima é o gargalo (10 Mbit/s), para que o ataque produza degradação mensurável dentro da capacidade de uma única máquina. O servidor HTTP entrega um arquivo de 20 KB e foi configurado sem *SYN cookies*, com fila de 1.024 conexões semiabertas e duas retransmissões de SYN-ACK, de modo que uma inundação SYN ocupa a fila por cerca

de 7 s. Um dos clientes legítimos, quando sorteado, faz *upload* contínuo de 2 Mbit/s com `iperf3`, representando tráfego legítimo de alta taxa. Como os *hosts* são conhecidos, as regras de encaminhamento e as entradas de contagem por endereço são instaladas de forma proativa, e uma regra de menor prioridade envia ao controlador apenas o tráfego de endereços desconhecidos. Assim, o controlador não depende de *packet-in* a cada novo fluxo.

5.3. Formulação como MDP

A decisão é tomada *por origem*, com uma única política compartilhada por todas as origens ativas. Essa escolha mantém a dimensão do estado fixa, independentemente do número de *hosts*, e permite reutilizar a experiência de todas as origens durante o treinamento.

Estado. Para cada origem i no intervalo t, a observação é

$$o_t^i = \left[\, x_t^i \parallel g_t \parallel \mathrm{onehot}(a_{t-1}^i) \parallel \tau_t^i \,\right], \tag{1}$$

em que $\parallel$ denota concatenação; x_t^i reúne atributos locais normalizados (taxa de pacotes e de bytes, tamanho médio de pacote, proporção de SYN, número estimado de conexões distintas, número de portas de destino distintas e razão entre tráfego recebido e enviado pela origem); g_t reúne atributos globais (utilização do enlace da vítima, latência de sondas enviadas ao servidor por um *host* de monitoramento, entropia dos IPs de origem calculada pelos contadores de pacotes, taxa de *packet-in* no controlador e ocupação do nó de análise); a_{t-1}^i é a ação vigente; e τ_t^i é o tempo desde a última mudança de ação. Atributos são calculados sobre janelas deslizantes de alguns intervalos para capturar ataques pulsados.

Ações. O conjunto $\mathcal{A} = \{a_0, a_1, a_2, a_3\}$ é graduado (Tabela 2). O bloqueio usa *hard_timeout*, de modo que um erro do agente se desfaz automaticamente; o redirecionamento envia o tráfego a um nó de análise com capacidade limitada (enlace de 5 Mbit/s), um roteador Linux que aplica limitação por origem com o módulo *hashlimit* do *iptables* (100 pacotes/s por origem) e encaminha o tráfego aprovado ao servidor com latência adicional. Soluções com estado, como o SYNPROXY, foram descartadas porque exigiriam caminho simétrico, e as respostas do servidor retornam diretamente aos clientes.

Tabela 2. Ações do agente e sua implementação em OpenFlow 1.3.

Ação	Implementação	Custo / efeito colateral
a_0: permitir	Remove regras restritivas da origem; encaminhamento normal	Nenhum, mas o ataque prossegue se a origem for maliciosa
a_1: limitar a taxa	*FlowMod* com instrução `meter` e banda *drop* de taxa r_{lim}	Usuário legítimo limitado sofre perda parcial
a_2: redirecionar	*FlowMod* que reescreve o MAC de destino e envia o tráfego pela porta do nó de análise	Latência adicional; capacidade de análise limitada
a_3: bloquear temporariamente	*FlowMod* de descarte com *hard_timeout* T_b	Usuário legítimo bloqueado fica indisponível por T_b

Recompensa. A recompensa da origem i combina um termo local e um termo global compartilhado:

$$r_t^i = \underbrace{w_1\,\mathbf{1}[i \in \mathcal{M}]\,b(a_t^i) - w_2\,\mathbf{1}[i \in \mathcal{L}]\,h(a_t^i)}_{\text{local}} + \underbrace{w_3\,D_t - w_4\,U_t^{\mathrm{atq}}}_{\text{global}} - w_5\,c(a_t^i, a_{t-1}^i), \tag{2}$$

em que $\mathcal{M}$ e $\mathcal{L}$ são os conjuntos de origens maliciosas e legítimas (conhecidos apenas no ambiente de treinamento); b é o benefício de aplicar a ação a um atacante, medido pelo efeito real (fração do seu tráfego efetivamente mitigada no intervalo, de modo que limitar um atacante que já envia abaixo de $r_{\lim}$ não rende recompensa), e h é o dano a um usuário legítimo, $h = \max(h_0(a),$ fração do tráfego afetada$)$, com piso $h_0 = (0;\ 0{,}2;\ 0{,}3;\ 1)$ para (permitir, limitar, redirecionar, bloquear). No treinamento, 30% dos episódios têm atacantes de baixa taxa (3% a 25% da nominal) e 30% dos clientes legítimos têm taxa até quatro vezes maior, para criar origens ambíguas. Separar b e h é necessário: com uma única severidade $\rho(a)$ multiplicando ambos os termos, o valor esperado local seria linear em ρ e o ótimo seria sempre uma ação extrema; com h convexo, as ações intermediárias passam a ser ótimas quando a origem é ambígua (por exemplo, com probabilidade de ataque 0,5, limitar ou redirecionar supera permitir e bloquear); D_t é a disponibilidade do serviço no intervalo (fração de requisições legítimas atendidas dentro do prazo); U_t^{atq} é a fração da capacidade do enlace da vítima ocupada por tráfego de ataque; e $c(\cdot)$ modela o custo operacional de instalar ou trocar regras, desestimulando oscilações. Com $w_2 > w_1$, um falso positivo custa mais que o benefício de restringir um atacante, refletindo o objetivo de não bloquear usuários legítimos. Os rótulos são usados apenas para compor a recompensa no treinamento; em operação, o agente decide somente a partir de o_t^i. Uma variante sem rótulos, que usa apenas os termos globais observáveis, será avaliada como ablação.

5.4. Agente e treinamento

O agente principal é um *Double DQN* [van Hasselt et al. 2016] com arquitetura *dueling* [Wang et al. 2016]: uma rede de duas camadas ocultas de 128 neurônios (ReLU), otimizador Adam, memória de repetição e política ε-gulosa, implementado em PyTorch. Como o bloqueio trava a origem por T_b intervalos, cada transição vai de uma decisão da origem até a sua próxima decisão, k intervalos depois, com recompensa descontada acumulada e alvo $r + \gamma^k \max_{a'} Q(s', a')$ (formulação semi-Markoviana). Para o controlador, os pesos são exportados e a inferência é feita em *numpy*, sem dependência de bibliotecas de aprendizado profundo. Os hiperparâmetros utilizados estão na Tabela 3.

Tabela 3. Hiperparâmetros do agente.

Parâmetro	**Valor**
Intervalo de decisão Δt	1 s
Fator de desconto γ	0,95
Taxa de aprendizado	10^{-4}
Memória de repetição / lote	10^5 transições / 64
Atualização da rede-alvo	a cada 1.000 passos
ε (exploração)	1,0 $\rightarrow$ 0,05 em 100.000 passos
Passos de gradiente (simulador)	300.000
Bloqueio temporário T_b	10 s
Taxa de limitação $r_{\lim}$	10% da capacidade do gargalo (1 Mbit/s)
Pesos da recompensa $w_1, \ldots, w_5$	1; 2; 0,5; 0,5; 0,05

Como cada passo no Mininet ocorre em tempo real ($\Delta t = 1$ s), treinar exclusivamente na emulação seria lento. Por isso, o treinamento ocorre em duas fases: (i) um ambiente Gymnasium [Towers et al. 2025] que simula, em modelo de fluidos, as origens,

o enlace gargalo, os medidores e o nó de análise, com aleatorização de intensidade, número de atacantes e perfis legítimos; e (ii) ajuste fino com episódios no próprio Mininet, com exploração $\varepsilon = 0{,}1$, reconstruindo as transições a partir dos registros de decisão do controlador. O simulador é calibrado com execuções sem defesa (B0) no Mininet: a taxa efetiva do gerador de ataque, os tamanhos de pacote e as razões de resposta são estimados diretamente dos atributos observados, e os parâmetros de dinâmica (tamanho e tempo de permanência da fila de conexões semiabertas e sensibilidade à perda no gargalo) são escolhidos por busca em grade, minimizando o erro da disponibilidade sem defesa por cenário. Ambas as fases usam apenas os cenários de treinamento (S0–S3, Tabela 4), assim como a escolha dos pesos da recompensa e dos limiares dos *baselines*. A avaliação final é feita *somente* no Mininet. O MininetGym [Finistrella et al. 2025] foi considerado como alternativa de integração, mas não adotado por depender do controlador OpenDaylight.

5.5. Baselines

- **B0 – Sem defesa**: encaminhamento normal.
- **B1 – Limiar fixo**: bloqueia temporariamente uma origem cuja taxa de pacotes excede um limiar calibrado no cenário normal.
- **B2 – Entropia**: detecção por queda da entropia, seguida de bloqueio temporário das origens cuja participação no tráfego excede $1/N$. [Mousavi and St-Hilaire 2015] usam a entropia dos endereços de destino; como nesta topologia todo o tráfego converge para um único servidor, adotou-se a entropia das *origens* do tráfego destinado à vítima, com limiar no percentil 1 do cenário normal.
- **B3 – Supervisionado**: Random Forest [Breiman 2001] treinado com os mesmos atributos x_t^i e tráfego rotulado coletado nos cenários de treinamento; bloqueia quando a probabilidade de ataque excede 0,5.
- **B4 – Supervisionado graduado**: o mesmo Random Forest, com faixas de probabilidade mapeadas para as quatro ações. Esse *baseline* isola o efeito do RL em relação ao simples uso de ações graduadas.

6. Planejamento Experimental

6.1. Questões de pesquisa e hipóteses

QP1 (eficácia). O agente reduz o tráfego de ataque que alcança a vítima com menos falsos positivos que os *baselines* B1–B4? *H1: o agente atinge taxa de bloqueio de ataque comparável à de B3 com taxa de falsos positivos significativamente menor que B1 e B3.*

QP2 (adaptação). A política mantém o desempenho sob padrões de tráfego e condições de rede não vistos no treinamento (ataques pulsados, *flash crowds*, ataques multivetor e enlaces com atraso e perda)? *H2: a degradação de disponibilidade do agente nesses cenários é menor que a dos limiares fixos.*

QP3 (ações graduadas). Qual é o ganho do espaço de quatro ações em relação a um agente com apenas {permitir, bloquear}? *H3: o espaço graduado reduz o dano colateral sem aumentar o tempo de reação.*

QP4 (custo). Qual é o tempo de reação e a sobrecarga que o agente impõe ao controlador? *H4: o ciclo de decisão (coleta, inferência e instalação de regras) permanece abaixo de Δt, e o tempo de reação é de poucos intervalos de decisão, comparável ao de B1.*

6.2. Cenários

A Tabela 4 descreve os cenários. O tráfego legítimo é gerado por clientes HTTP com chegadas de Poisson, tráfego de fundo com `iperf3` e sondas ICMP de latência; os ataques são gerados com `hping3`. Cada execução dura 180 s: 30 s de aquecimento, 120 s de ataque e 30 s de recuperação. Cenários marcados como *não vistos* não são usados no treinamento.

Tabela 4. Cenários experimentais.

Id	Cenário	Descrição	Treino?
S0	Normal	Apenas tráfego legítimo (mede falsos positivos de base)	Sim
S1	*SYN flood*	3 intensidades (baixa, média, alta), 2 a 4 atacantes	Sim
S2	*UDP flood*	3 intensidades	Sim
S3	*ICMP flood*	3 intensidades	Sim
S4	Ataque pulsado	Rajadas *on-off* de UDP/SYN (baixa taxa média)	Não
S5	*Flash crowd*	Aumento abrupto de clientes legítimos, sem ataque	Não
S6	Multivetor	SYN + UDP simultâneos, com mais atacantes que no treino	Não
S7	Rede degradada	*SYN flood* de intensidade média com 20 ms de atraso e 1% de perda nos enlaces de borda (`netem`)	Não
S8	Atacantes furtivos	SYN/UDP de baixa taxa (3% a 25% da intensidade baixa) e clientes legítimos intensos, gerando origens ambíguas	Não

6.3. Métricas

Seja $B_{\mathrm{env}}^{\mathrm{atq}}$ o volume de ataque enviado e $B_{\mathrm{mit}}^{\mathrm{atq}}$ o volume de ataque descartado pelas regras de mitigação (medidores, regras de descarte e nó de análise), obtido dos contadores dessas regras. Descartes causados apenas pelo congestionamento do gargalo não contam como mitigação. As métricas são:

- **Taxa de ataque bloqueado**: $\mathrm{TAB} = B_{\mathrm{mit}}^{\mathrm{atq}} / B_{\mathrm{env}}^{\mathrm{atq}}$.
- **Usuários legítimos bloqueados (falsos positivos)**: fração de origens legítimas que receberam ação restritiva em algum intervalo, e fração do tempo em que permaneceram restritas, reportando separadamente limitação/bloqueio e redirecionamento, já que este último apenas acrescenta latência.
- **Disponibilidade**: fração de requisições HTTP legítimas concluídas com sucesso em até 2 s.
- **Latência e perda**: tempo de resposta HTTP (mediana e p95), RTT das sondas e perda de pacotes legítimos.
- **Tempo de reação**: intervalo entre o início do ataque e a primeira ação restritiva efetiva sobre cada atacante; e tempo até a disponibilidade voltar a 95% do valor medido no cenário S0.
- **Sobrecarga no controlador**: uso de CPU e memória do processo do controlador, mensagens OpenFlow por segundo (*FlowMod*, *MeterMod*, estatísticas), tempo de inferência por decisão e número de regras instaladas.

6.4. Repetições e controle de variáveis

Cada combinação de método e cenário foi executada com 3 sementes distintas e fixadas (o protocolo prevê até 10), reiniciando a rede entre execuções; todos os métodos usam as mesmas sementes, topologia, perfis de tráfego e intervalo de coleta. Os resultados são reportados como média e intervalo de confiança de 95% (*bootstrap*). Para comparar o agente com cada *baseline*, aplica-se o teste de postos sinalizados de Wilcoxon pareado por cenário e semente, com correção de Holm para comparações múltiplas. Todos os resultados, inclusive os desfavoráveis, são reportados, e os dados brutos (estatísticas, registros de decisão e *logs* dos clientes) são preservados. Com seis métodos, 15 configurações de cenário (S1–S3 em três intensidades cada) e execuções de 3 min, a campanha principal soma cerca de 14 horas de emulação, executadas em lote por *scripts*; as ablações foram restritas aos cenários S1H, S4, S5 e S8.

7. Resultados

Estado da avaliação. Os resultados a seguir são *preliminares*: correspondem às primeiras 48 das 270 rodadas da campanha principal (180 s por rodada, semente 1001), cobrindo os cenários S0–S2 e S3L para os seis métodos. Os demais cenários de ICMP, os cenários de adaptação (S4–S8) e as ablações ainda estão em execução. Com uma única semente, os intervalos de confiança refletem apenas a variação entre cenários, e os testes de Wilcoxon pareados com correção de Holm não atingem significância em nenhuma comparação. O agente avaliado passou pelo ajuste fino no Mininet (12 episódios, $\epsilon = 0{,}1$) com a recompensa baseada em efeito.

7.1. Linha de base sem defesa e calibração

Sem defesa (B0, 180 s), a disponibilidade foi de 100% em S0; 99,9%, 93,7% e 7,8% em S1L/M/H; 99,9%, 21,1% e 1,9% em S2L/M/H; e 99,9%, 98,8% e 20,1% em S3L/M/H. A calibração com os três tipos de ataque reduziu o erro quadrático da disponibilidade simulada para 0,0175.

7.2. Eficácia e dano colateral no Mininet (QP1)

Tabela 5. Resultados preliminares no Mininet, média dos cenários S1L–S3L (180 s, uma semente; ± meia-largura do IC 95% por *bootstrap* entre cenários). TAB: tráfego de ataque descartado por limitação ou bloqueio (não inclui o redirecionado); Legít.: fração de origens legítimas que receberam alguma ação restritiva.

Método	TAB (%)	Legít. (%)	Disp. (%)	Lat. p95 (ms)	Reação (s)
B0 – Sem defesa	0,0	0,0	65,0 ± 28,2	921	–
B1 – Limiar fixo	94,2 ± 0,7	0,0	88,3 ± 12,9	673	1,7
B2 – Entropia	93,9 ± 0,8	100,0	24,0 ± 6,8	1.080	2,0
B3 – RF	94,6 ± 0,5	3,6 ± 3,6	88,0 ± 13,8	666	1,0
B4 – RF graduado	95,1 ± 0,7	0,0	88,9 ± 13,0	547	1,1
Agente (proposto)	52,0 ± 22,5	41,1 ± 23,2	67,4 ± 24,9	562	0,3

Os *baselines* B1, B3 e B4 descartaram cerca de 94–95% do tráfego de ataque quase sem restringir usuários legítimos e elevaram a disponibilidade média de 65,0% para cerca de 88% (Tabela 5). B2 (entropia) restringiu todas as origens legítimas, efeito

Tabela 6. Agente por cenário no Mininet (uma semente). Tempo restr.: fração do tempo de ataque em que as origens legítimas estiveram sob ação restritiva.

Cenário	TAB (%)	Legít. (%)	Tempo restr. (%)	Disp. agente (%)	Disp. B0 (%)
S0 (normal)	–	8,3	1,8	99,2	100,0
S1L (SYN)	80,7	62,5	3,3	32,4	99,9
S1M (SYN)	93,7	87,5	9,3	20,5	93,7
S1H (SYN)	49,6	87,5	7,4	20,4	7,8
S2L (UDP)	39,6	12,5	0,7	99,6	99,9
S2M (UDP)	13,7	12,5	0,6	99,5	21,1
S2H (UDP)	7,3	12,5	0,6	99,6	1,9
S3L (ICMP)	79,6	12,5	0,6	99,7	99,9

do limiar estimado sobre um tráfego normal de baixa entropia. O agente reagiu mais rápido que todos (0,3 s), mas, na média, ficou próximo da linha sem defesa.

A média esconde dois comportamentos opostos (Tabela 6). Sob UDP, o agente manteve 99,5–99,6% de disponibilidade, inclusive em S2H, onde sem defesa ela cai para 1,9%, embora a TAB tenha ficado entre 7% e 40%. A diferença se explica pela ação escolhida: em cerca de 95% dos intervalos de decisão sobre atacantes, o agente *redirecionou* o tráfego para o nó de análise, que o retira do caminho da vítima mas não é contado como descarte pela TAB. Sob SYN, ao contrário, a disponibilidade caiu para 20–32%, abaixo da linha sem defesa em S1L e S1M. O agente restringiu, em algum momento, de 62% a 88% das origens legítimas, quase sempre com limitação de taxa e por pouco tempo (3–9% do período de ataque); essa restrição curta, sozinha, não parece explicar a queda, cuja causa ainda está sendo investigada (ver Seção 8). Em S0, sem ataque, o agente restringiu 8% das origens legítimas, mantendo 99,2% de disponibilidade.

7.3. Adaptação e uso das ações (QP2 e QP3)

Os cenários de adaptação (S4–S8) ainda não foram executados no Mininet. No simulador calibrado, o agente obteve 97% de disponibilidade no ataque pulsado (S4), contra 93,5% de B1 e B3, e a variante {permitir, bloquear} chegou a 100%. A Tabela 7 mostra a distribuição das ações no simulador com origens ambíguas.

Tabela 7. Distribuição das ações durante o ataque no simulador com atacantes furtivos (40 episódios).

Política	Origens	Permitir	Limitar	Redirecionar	Bloquear
Agente (4 ações)	atacantes comuns	9,5%	2,7%	3,6%	84,3%
	atacantes furtivos	12,1%	0%	6,9%	81,0%
	legítimas	100%	0%	0%	0%
Agente {permitir, bloquear}	atacantes furtivos	13,5%	–	–	86,5%
B4 – RF graduado	atacantes furtivos	99,7%	0,3%	0%	0%

No simulador, as ações intermediárias foram pouco usadas (cerca de 6% das decisões sobre atacantes). No Mininet, depois do ajuste fino, o quadro se inverteu: o redirecionamento passou a ser a ação dominante sobre atacantes (cerca de 95% das decisões) e a limitação, a única restrição frequente sobre legítimos. O ajuste fino, portanto, alterou de

forma substancial a política, e as ações graduadas passaram a ser usadas; se esse uso traz vantagem (H3) depende das ablações, ainda em execução.

7.4. Custo no controlador (QP4)

Tabela 8. Custo no controlador no cenário S1H (180 s).

Método	CPU (%)	Memória (MB)	Msgs OF/s	Decisão (ms)
B1 – Limiar fixo	1,1	195	15,0	0,10
B3 – RF	3,9	198	15,1	27,94
Agente	1,0	68	15,2	0,29

O ciclo de decisão do agente levou cerca de 0,3 ms, três ordens de grandeza abaixo de $\Delta t = 1$ s, com CPU equivalente à do limiar fixo e menos memória; o Random Forest foi o mais custoso (cerca de 28 ms por ciclo). A hipótese H4 foi confirmada.

8. Discussão e Ameaças à Validade

Discussão. Os resultados preliminares dependem do tipo de ataque. Contra inundações volumétricas (UDP e ICMP), o agente protegeu a vítima tão bem quanto os *baselines*, reagiu mais rápido e afetou pouco os legítimos, preferindo redirecionar a bloquear. Contra inundação SYN, porém, foi pior que os *baselines* e, em dois dos três níveis, pior que não defender (H1 e H2 não confirmadas até aqui). Três explicações são plausíveis e serão verificadas. (i) Redirecionar um SYN *flood* protege menos que bloqueá-lo: as conexões semiabertas já acumuladas continuam ocupando a fila do servidor, e a recompensa por efeito trata redirecionamento e bloqueio de modo parecido. (ii) Lacuna entre simulação e emulação: sob SYN, as conexões legítimas que não completam o *handshake* alteram as *features* dos clientes (razão SYN/ACK, pacotes por fluxo) de forma não reproduzida no simulador, o que leva o agente a limitá-los. (iii) O ajuste fino foi curto (12 episódios) e com exploração ($\epsilon = 0{,}1$), o que pode ter reforçado escolhas feitas ao acaso. A comparação também expõe uma limitação da TAB, que subestima a mitigação de quem redireciona; por isso a disponibilidade é a métrica principal desta análise. Por fim, os *baselines* supervisionados mostram que as inundações do `hping3` são facilmente separáveis, de modo que bloquear rapidamente já é uma resposta muito boa.

Validade interna. O agente foi treinado no simulador e ajustado com poucos episódios no Mininet. O Mininet compartilha a CPU da máquina hospedeira, o que pode afetar latência e custo; todos os métodos foram executados nas mesmas condições. A recompensa usa rótulos disponíveis apenas na emulação.

Validade externa. A topologia é pequena, há um único controlador e o tráfego é sintético; os resultados podem não se generalizar para redes de produção, ataques com falsificação de endereços (*IP spoofing*) ou ataques à camada de aplicação, que estão fora do escopo.

Validade de construto. A disponibilidade é medida por requisições HTTP com prazo fixo, e o falso positivo é definido no nível de origem: uma limitação breve conta tanto quanto um bloqueio longo, razão pela qual também se reporta a fração do tempo sob restrição. A TAB não conta o tráfego redirecionado.

Validade de conclusão. Há uma única semente e só os cenários S0–S2 e S3L; nenhuma diferença é estatisticamente significativa após a correção de Holm. As conclusões são indicativas e dependem da campanha completa, em execução.

Aspectos éticos. Todo tráfego de ataque é gerado em ambiente emulado e isolado, sem atingir redes ou serviços reais.

9. Conclusão

Este trabalho propôs e implementou um agente de aprendizado por reforço profundo (Double Dueling DQN) para mitigação adaptativa de ataques DDoS em SDN, que escolhe, para cada origem de tráfego, entre permitir, limitar a taxa, redirecionar para análise ou bloquear temporariamente. A solução completa – controlador os-ken com contadores por origem e sFlow, simulador calibrado com traços do Mininet, cinco *baselines*, ajuste fino na emulação e *scripts* reprodutíveis de experimento e análise – foi construída sobre ferramentas abertas. Nos resultados preliminares, o agente foi o mais rápido (0,3 s), teve custo desprezível no controlador e, contra inundações UDP e ICMP, manteve a disponibilidade acima de 99% usando sobretudo o redirecionamento. Contra inundação SYN, porém, restringiu muitos usuários legítimos e ficou atrás do limiar fixo e do Random Forest, que tiveram o melhor desempenho geral.

Como trabalhos futuros, destacam-se: concluir a campanha com três sementes, os cenários de adaptação (S4–S8) e as ablações; diagnosticar a queda sob SYN, diferenciando na recompensa o efeito do redirecionamento e do bloqueio sobre a fila de conexões; modelar no simulador o efeito do SYN sobre as *features* dos clientes legítimos; fazer um ajuste fino mais longo; contabilizar o tráfego redirecionado na métrica de mitigação; e estender a proposta para múltiplos controladores e ataques com falsificação de endereços.

Referências

Albuquerque Jr., F. d. A. C. d. and Fonseca, I. E. (2023). Mitigando a ameaça dos ataques slow DDoS a redes SDN usando consolidação de regras. In *Anais do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg)*, pages 293–306, Porto Alegre. SBC. DOI: 10.5753/sbseg.2023.232808.

Bahashwan, A. A., Anbar, M., Manickam, S., Al-Amiedy, T. A., Aladaileh, M. A., and Hasbullah, I. H. (2023). A systematic literature review on machine learning and deep learning approaches for detecting DDoS attacks in software-defined networking. *Sensors*, 23(9):4441. DOI: 10.3390/s23094441.

Biondi, P. and Scapy Community (2026). Scapy: Packet crafting for Python. `https://scapy.net`. Versão 2.8.

Braga, R., Mota, E., and Passito, A. (2010). Lightweight DDoS flooding attack detection using NOX/OpenFlow. In *IEEE 35th Conference on Local Computer Networks (LCN)*, pages 408–415. DOI: 10.1109/LCN.2010.5735752.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32. DOI: 10.1023/A:1010933404324.

Chen, S., Shen, C., Wu, C., and Shen, Y. (2022). DeepThrottle: Deep reinforcement learning for router throttling to defend against DDoS attack in SDN. In *IEEE International*

Performance, Computing, and Communications Conference (IPCCC), pages 416–417. DOI: 10.1109/IPCCC55026.2022.9894298.

Dias, V., Silva, M., Gomes, M., Oliveira, L. B., Abreu, D., Ferreira, J., and Abelém, A. (2023). Detecção de ataques DDoS em redes SDN utilizando aprendizado de máquina: Uma abordagem em microsserviços. In *Anais Estendidos do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg)*, pages 141–152, Porto Alegre. SBC. DOI: 10.5753/sbseg_estendido.2023.234287.

Elsayed, M. S., Le-Khac, N.-A., and Jurcut, A. D. (2020). InSDN: A novel SDN intrusion dataset. *IEEE Access*, 8:165263–165284. DOI: 10.1109/ACCESS.2020.3022633.

Feng, Y., Li, J., and Nguyen, T. (2020). Application-layer DDoS defense with reinforcement learning. In *IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*, pages 1–10. DOI: 10.1109/IWQoS49365.2020.9213026.

Finistrella, S., Mariani, S., and Zambonelli, F. (2025). MininetGym: A modular SDN-based simulation environment for reinforcement learning in cybersecurity. *SoftwareX*, 31:102312. DOI: 10.1016/j.softx.2025.102312.

Khozam, S., Blanc, G., Tixeuil, S., and Totel, E. (2024). DDoS mitigation while preserving QoS: A deep reinforcement learning-based approach. In *IEEE 10th International Conference on Network Softwarization (NetSoft)*, pages 369–374. DOI: 10.1109/NetSoft60951.2024.10588889.

Khozam, S., Blanc, G., Tixeuil, S., and Totel, E. (2025). QoSentry: A reinforcement learning framework for QoS-preserving DDoS mitigation in software-defined networks. *Journal of Network and Systems Management*, 33:97. DOI: 10.1007/s10922-025-09971-8.

Kreutz, D., Ramos, F. M. V., Veríssimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76. DOI: 10.1109/JPROC.2014.2371999.

Kuzmanovic, A. and Knightly, E. W. (2003). Low-rate TCP-targeted denial of service attacks: The shrew vs. the mice and elephants. In *Proceedings of the ACM SIGCOMM 2003 Conference*, pages 75–86. DOI: 10.1145/863955.863966.

Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, pages 1–6. DOI: 10.1145/1868447.1868466.

Liu, Y., Dong, M., Ota, K., Li, J., and Wu, J. (2018). Deep reinforcement learning based smart mitigation of DDoS flooding in software-defined networks. In *IEEE 23rd International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)*, pages 1–6. DOI: 10.1109/CAMAD.2018.8514971.

Malialis, K. and Kudenko, D. (2013). Multiagent router throttling: Decentralized coordinated response against DDoS attacks. In *Proceedings of the 27th AAAI Conference on Artificial Intelligence*, pages 1551–1556. DOI: 10.1609/aaai.v27i2.19000.

Malialis, K. and Kudenko, D. (2015). Distributed response to network intrusions using multiagent reinforcement learning. *Engineering Applications of Artificial Intelligence*, 41:270–284. DOI: 10.1016/j.engappai.2015.01.013.

McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2):69–74. DOI: 10.1145/1355734.1355746.

Mirkovic, J. and Reiher, P. (2004). A taxonomy of DDoS attack and DDoS defense mechanisms. *ACM SIGCOMM Computer Communication Review*, 34(2):39–53. DOI: 10.1145/997150.997156.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533. DOI: 10.1038/nature14236.

Mousavi, S. M. and St-Hilaire, M. (2015). Early detection of DDoS attacks against SDN controllers. In *International Conference on Computing, Networking and Communications (ICNC)*, pages 77–81. DOI: 10.1109/ICCNC.2015.7069319.

Open Networking Foundation (2012). OpenFlow switch specification, version 1.3.0 (Wire protocol 0x04). ONF TS-006.

Pfaff, B., Pettit, J., Koponen, T., Jackson, E., Zhou, A., Rajahalme, J., Gross, J., Wang, A., Stringer, J., Shelar, P., Amidon, K., and Casado, M. (2015). The design and implementation of Open vSwitch. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 117–130.

Ryu SDN Framework Community (2020). Ryu: Component-based software defined networking framework. `https://github.com/faucetsdn/ryu`. Versão 4.34; fork mantido: os-ken (`https://opendev.org/openstack/os-ken`).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

Sharafaldin, I., Lashkari, A. H., Hakak, S., and Ghorbani, A. A. (2019). Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. In *International Carnahan Conference on Security Technology (ICCST)*, pages 1–8. DOI: 10.1109/CCST.2019.8888419.

Shen, C., Gao, W., Xu, Q., Zou, T., Zhu, J., and Zhang, R. (2023). Fast DDoS traffic throttling and normal traffic permitting in SDN-IoT: A deep reinforcement learning approach. In *IEEE International Conference on Communications (ICC)*, pages 353–358. DOI: 10.1109/ICC45041.2023.10279685.

Silva, R. R., Henriques, F. d. R., Moraes, I. M., and Mascarenhas, D. M. (2024). Detecção de ataques de negação de serviço distribuídos com algoritmos de aprendizado de máquina. In *Anais do XXIV Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg)*, pages 226–241, Porto Alegre. SBC. DOI: 10.5753/sbseg.2024.241684.

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition.

Tebbaa, K., Chakir, O., Maleh, Y., and Belaissaoui, M. (2025). Mitigating DDoS attacks in software-defined networks: A systematic literature review of machine learning and deep learning approaches. *Iran Journal of Computer Science*, 9:5. DOI: 10.1007/s42044-025-00386-x.

Towers, M., Kwiatkowski, A., Balis, J., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Terry, J., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., and Younis, O. G. (2025). Gymnasium: A standard interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, pages 163114–163129.

van Hasselt, H., Guez, A., and Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*. DOI: 10.1609/aaai.v30i1.10295.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *PMLR*, pages 1995–2003.

Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4):279–292. DOI: 10.1007/BF00992698.

Yan, Q., Yu, F. R., Gong, Q., and Li, J. (2016). Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: A survey, some research issues, and challenges. *IEEE Communications Surveys & Tutorials*, 18(1):602–622. DOI: 10.1109/COMST.2015.2487361.

Yungaicela-Naula, N. M., Vargas-Rosales, C., and Pérez-Díaz, J. A. (2023). SDN/NFV-based framework for autonomous defense against slow-rate DDoS attacks by using reinforcement learning. *Future Generation Computer Systems*, 149:637–649. DOI: 10.1016/j.future.2023.08.007.

Yungaicela-Naula, N. M., Vargas-Rosales, C., Pérez-Díaz, J. A., and Carrera, D. F. (2022). A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning. *Journal of Network and Computer Applications*, 205:103444. DOI: 10.1016/j.jnca.2022.103444.